

Programmable In-Network Timing Obfuscation for DNP3

Anonymous Submission

Abstract too long

Abstract—Adversaries preparing attacks on industrial control systems (ICSs) such as power grids first perform reconnaissance, and device fingerprinting supplies much of the knowledge they need. In ICS networks, the fingerprint does not have to come from the payload. The interval between the transport acknowledgment of an outstation and its application response reflects the execution time of the device, and it identifies both the device type and the operation the device performs. Existing traffic obfuscation does not remove this interval, because it reshapes packet sizes and inter-packet gaps of encrypted flows, whereas ICS traffic is unencrypted and any padding or dummy packet reveals itself.

To overcome these challenges, we propose an in-network timing-obfuscation framework that replaces the interval with a value the operator configures. A programmable switch placed in front of the outstation holds the acknowledgment and the response and releases each of them at a deadline computed from the same instant, such that the execution time of the device cancels from the interval an adversary measures. The switch only delays the packets it receives, so both endpoints run unmodified and no DNP3 byte changes. We implement the framework on an Intel Tofino-1 switch and evaluate it against a physical SEL-751A protective relay over 63,360 DNP3 exchanges. The experimental results show that, with negligible overhead, the framework can disrupt timing-based fingerprinting. The released interval settles on the configured value, and the mutual information it carries about the operation falls into its permutation null. A classifier trained on Timing OFF traffic falls from 0.745 balanced accuracy to three-class chance, and an adaptive adversary that pools twenty exchanges stays at chance, where Timing OFF traffic lets it reach 0.988. The framework adds no frame and no byte, and its 22.7 ms of added latency stays well within the 201 ms after which the master repeats an unacknowledged request.

Index Terms—traffic obfuscation, device fingerprinting, DNP3, programmable switches.

I. INTRODUCTION

Device fingerprinting has been an essential step in cyber reconnaissance, allowing adversaries to reveal unique features of target networks and to design effective, stealthy attack strategies. These techniques are becoming increasingly critical in industrial control systems (ICSs) such as power grids, where adversaries often use IP-based control networks and computing devices within as entry points to inflict physical damage. Consequently, fingerprinting shifts the focus from visited websites and user biometric behavior to device models and types of control operations that are critical to ICS attacks.

In the 2015 attack that disrupted Ukrainian power grids and the Stuxnet attack that disrupted Iranian nuclear power facilities, it is widely believed that adversaries stay in their systems for at least 6 months to perform cyber reconnaissance [1], [2].

To disrupt device fingerprinting, many studies have presented network traffic obfuscation [3], [4]. Device fingerprinting targeting general computing environments generally relies on network-level features such as packet size and/or inter-packet latency observed from communication patterns [5]. Consequently, existing traffic obfuscation often focuses on (i) padding and splitting network packets, which hide or change the distribution of network packet sizes [3]; and (ii) delaying network packets and adding dummy ones, which disrupt the inter-packet timing pattern [4], [6]. Since manipulating communication networks can introduce runtime overhead, recent works have begun to offload traffic obfuscation onto programmable network switches, which change communication patterns at much higher line rates than CPUs [7]–[9].

Unfortunately, these methods cannot be directly applied to ICS environments for two main reasons. First, device fingerprinting methods on ICS devices rely on different features from the ones in general computing environments. Instead of using network layer knowledge, they attempt to use behavior on physical devices to reveal device model or types of operations. For example, work in [10] uses the latency between the TCP acknowledge packets and the actual response from the same device to estimate execution time there. Since each vendors may choose different materials or algorithms to implement the same control functionality, this time can be accurate in fingerprinting those devices. Second, existing traffic obfuscation is performed over encrypted channels, which are necessary to mix the obfuscated and normal traffic. Unfortunately, ICS networks still rely on unencrypted traffic due to a wide range of legacy devices, despite ICS network protocols may provide security features. Directly padding or adding dummy packets can easily reveal obfuscated packets, exposing the the underlying traffic pattern to adversaries.

In this paper, we propose an in-network timing-obfuscation framework for ICS traffic. Instead of padding packets or adding dummy ones, the framework changes only when the packets of a transaction leave the network, such that the timing an adversary measures becomes a value the operator configures rather than a property of the device.

A. Contributions

To overcome the two drawbacks above, we design a framework that runs on a programmable switch placed in front of

an outstation. The switch holds the transport acknowledgment and the application response of the outstation, and it releases each of them at a deadline computed from the moment the acknowledgment arrives. Because both deadlines are computed from the same instant, the execution time of the device cancels from the interval between the two releases. Because the switch forwards the same packets it received, the obfuscated traffic carries nothing that marks it as obfuscated, which is what a plaintext protocol requires.

The Framework’s Role. We position the framework as a complement to the protections an ICS already runs, such as intrusion detection. It reads no payload and changes no byte, and it requires no modification of the master, the outstation or the DNP3 protocol. Consequently, it can protect legacy devices that can neither be patched nor replaced, which are the devices that fingerprinting targets.

The main contributions of the paper are:

- **Replace the execution time of a device with a configured value.** We design a release schedule that holds real protocol packets. It computes both deadlines from one instant, so the execution time of the outstation cancels from the released interval. To the best of our knowledge, this is the first work that replaces the cross-layer response time of an ICS device inside the network. Existing defenses mimic, randomize or encrypt what the device sends.
- **Obfuscate plaintext ICS traffic without adding a packet.** The framework only delays packets, so it needs no encryption to hide its operation. The operator bounds the added delay by choosing the release budget, and that bound keeps every exchange within the timers of TCP and DNP3.
- **Implement the framework at line rate.** On an Intel Tofino-1 switch, we hold each packet behind a strict-priority queue of blocker packets the switch generates. The queue sustains a hold until its deadline without a timer. We cover both the read and the control transaction of DNP3.

For evaluation, we placed the switch between a DNP3 master and a physical SEL-751A feeder protection relay and collected 63,360 exchanges, half of them with the framework running. We tested the released interval against (i) a classifier trained before the framework was deployed, (ii) an adversary that retrains on obfuscated traffic, and (iii) an adversary that pools up to twenty exchanges into one signature, as Formby et al. do.

Our experimental results show that the framework disrupts timing-based fingerprinting while introducing negligible overhead:

- The released interval settles on the configured value. Its interquartile range falls from 2.778 to 0.006 ms, and the operator can place it anywhere from 1 to 22 ms at a fixed cost.
- The interval no longer identifies the operation. Its mutual information with the transaction class falls from 0.383 to

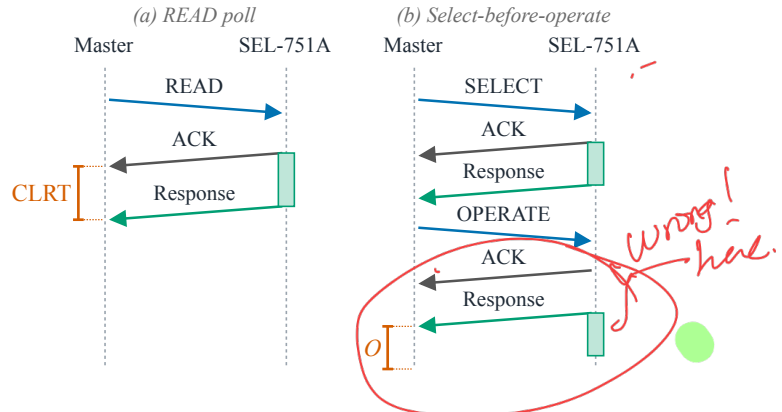


Fig. 1. The two DNP3 transactions, observed on the master-facing link. (a) A READ poll, whose CLRT runs from the acknowledgment to the response. (b) A select-before-operate control, whose master-visible interval O runs from the acknowledgment of the OPERATE to its response. The shaded bar marks the outstation producing the answer.

0.004 bits, inside the permutation null. A classifier trained on Timing OFF traffic falls from 0.745 balanced accuracy to chance, and an adaptive adversary that pools twenty exchanges stays at chance, against 0.988 on Timing OFF traffic.

- The overhead is negligible. The framework adds no frame and no byte, the median response time rises by 22.7 ms, well within the 201 ms after which the master repeats an unacknowledged request, and no exchange was retransmitted.

II. BACKGROUND AND MOTIVATION

In this section, we first present the DNP3 transactions the framework protects. Then we explain why the timing of these transactions identifies a device and how a programmable switch can change that timing.

A. DNP3 Read and Control Transactions

DNP3 carries supervisory control and data acquisition traffic over TCP between a master and an outstation [11]. In Figure 1, we show the two transactions the framework protects. The master polls the outstation with a READ request, function code 1, and the outstation returns the requested measurements in a response, function code 0x81. A control command instead uses select-before-operate. The master sends a SELECT, function code 3, that names an output point, and the outstation confirms it. The master then sends an OPERATE, function code 4, which the outstation executes and confirms with a solicited response.

Because DNP3 runs over TCP, the transport layer of the outstation acknowledges each request. On the relay we evaluate, the acknowledgment travels in a segment of its own before the application answers, although TCP also allows it to ride on the response [12]. Each exchange therefore appears on the wire as three packets: a request, a transport acknowledgment carrying no DNP3 payload, and a DNP3 response. The framework protects exchanges of this form and forwards any other exchange unchanged.

We limit the framework to these function code.

we have to define Physical operation time for SBO

Definition 1. Cross-Layer Response Time (CLRT): the interval between the arrival of the transport acknowledgment and the arrival of the DNP3 response of the same exchange, measured on the link to the master.

B. Execution Time as a Fingerprint

An outstation answers a request twice, from two different layers. Its TCP stack acknowledges the request as soon as it arrives, whereas its application responds only after the device has done the requested work. Because both packets travel the same path in the same direction, the common propagation delay cancels. Consequently, the CLRT is dominated by the execution time of the outstation.

Execution time differs between device models, since vendors implement the same function with different hardware and algorithms, and it differs between the operations one device performs. Formby et al. classify DNP3 device types and models by the distribution of the CLRT [10], and Lin et al. measured execution times that differ across devices from three vendors, one of which is the SEL-751A we evaluate [13]. The same separation holds between the operations of one device. For example, our relay takes 2.937 ms to answer a control command and 2.116 ms to answer a poll, while it acknowledges the two requests 0.003 ms apart. **The function code already names the operation in plaintext**, and the CLRT adds how this device performs it, which is the profile a fingerprinting adversary builds.

C. Holding Packets in a Programmable Switch

A programmable switch exposes a match-action pipeline, written in P4 [14], that parses headers, keeps state in registers and places packets into output queues at line rate. Each egress port offers several queues served in strict priority, a primitive that can also approximate programmable scheduling [15]. Because such a pipeline can delay a packet and release it later without a host in the loop, a switch between the master and the outstation can hold the acknowledgment and the response of a transaction and release them on a schedule the operator sets. As such, the switch can change the timing an adversary observes without changing either endpoint or any byte of the payload.

III. THREAT MODEL AND OBJECTIVES

In this section, we first describe the adversary we consider and the knowledge it seeks. Then we specify the three research objectives the framework must achieve, RO1 to RO3, and the attacks it does not address. **scope of our work.**

A. Threat Model

Assumptions on Adversary's Capability. We assume that an adversary has penetrated the control network and passively monitors the master-facing link, the position from which Formby et al. fingerprint outstations [10], as shown in Figure 2. We assume that the adversary **is able to** records the headers, the size and the arrival time of every packet. Because DNP3 carries no transport security by default [16], the adversary also

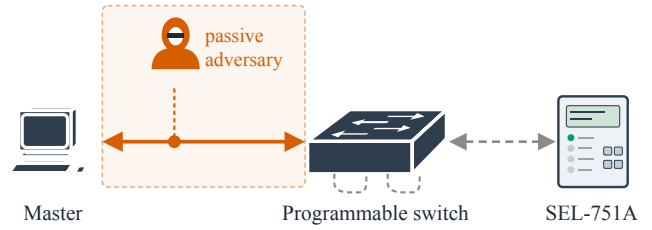


Fig. 2. Observation model. The adversary monitors the master-facing link (shaded). The relay-facing link and the lanes inside the switch (dashed) are not visible to the adversary.

reads the function code of every request and can label each exchange by its operation, e.g., a READ poll or the SELECT and OPERATE of a control command. We assume that the adversary knows the framework and the interval it enforces, but cannot read the internal state of the switch or monitor the link between the switch and the outstation. **We argue that** this is a practical assumption, as the switch sits at the edge of the outstation, and the relay-facing link can be a single cable between two devices in the same substation cabinet.

Adversary's Goal. The adversary aims at fingerprinting the outstation from the timing of its exchanges, and from that timing it infers the device type, and the operation each exchange performs. Based on this knowledge, the adversary can design targeted attacks, e.g., false data injection that must match the devices it compromises [17]. We consider two adversaries. A *fixed* adversary builds its classifier from traffic captured before the framework is deployed and applies it afterwards. An *adaptive* adversary collects traffic after deployment, labels it with the plaintext function codes and retrain its classifier. The adaptive adversary can also pool many exchanges of the same operation into one signature, as Formby et al. do [10].

B. Research Objectives

The Framework's Objective. We aim at disrupting timing-based fingerprinting of an outstation without changing the outstation, the master or the DNP3 protocol. We specify three research objectives (RO) as follows:

- **RO1:** for both the read poll and the select-before-operate control, we aim at making the CLRT the adversary measures a value the operator configures, independent of the execution time of the outstation.
- **RO2:** for fixed and adaptive adversaries, we aim at removing the information the CLRT carries about the outstation, such that their classifiers perform no better than chance, even when they pool many exchanges.
- **RO3:** for the operation of the control network, we aim at negligible overhead. The framework adds no packet and no byte, and it keeps every added delay within the timers of TCP and DNP3, such that no request is retransmitted and no control operation is refused.

For Payload-Based Fingerprinting. We designed the framework to hide timing, not content. It forwards every packet at its original size and leaves the function codes readable, so

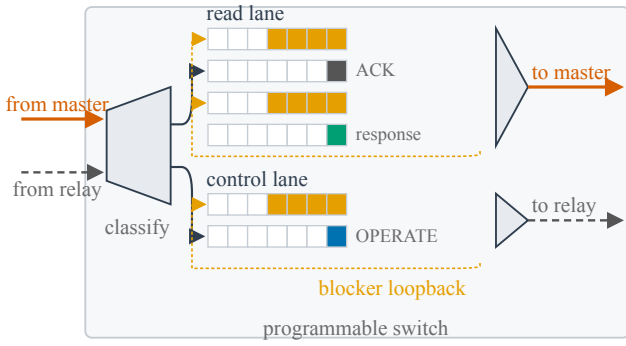


Fig. 3. Design of the framework on one switch pipeline. Ingress classifies each transaction and computes its release deadlines. The read lane holds what the outstation returns, for READ and for both phases of a control. The control lane holds the outgoing OPERATE. Each held packet waits behind a queue of switch-generated blocker packets (amber), which recirculate on a loopback port of their own. Solid lines are master-facing and observed by the adversary. Dashed lines are not, as in Figure 2.

the adversary still knows which operation an exchange carries. What the framework removes is how long the device takes to perform it. Formby et al. also estimate the physical operation time of a breaker from an event timestamp inside the response payload [10]. Such content is a different problem, and payload protection is complementary to the framework. We leave the combination of the two to future work.

For Active Adversaries. An adversary that injects, delays or drops packets, or that compromises the switch or the outstation, performs attacks beyond reconnaissance and is outside this model. Such an adversary still faces existing defenses, e.g., specification-based intrusion detection for DNP3 [18], and the framework leaves every byte those defenses inspect unchanged.

IV. DESIGN

In this section, we first present an overview of the framework. Then we describe how it replaces the CLRT of a read transaction (Section IV-B) and of a control transaction (Section IV-C), each followed by its security argument. Last, we argue why its overhead is negligible and bounded (Section IV-D).

A. Design Overview

In Figure 3, we present the design of the framework. We place a programmable switch between the master and the outstation, and the switch classifies every DNP3 exchange by its function code. Based on that classification, it holds the packets that carry the timing of the outstation and releases each of them at a deadline, such that the interval the adversary measures becomes the value the operator configures. We handle two kinds of transactions in two lanes.

Read Lane. For a READ poll and for the SELECT phase of a control command, the framework holds the transport acknowledgment and the application response of the outstation on their way to the master. It computes both release deadlines when the acknowledgment arrives (Section IV-B).

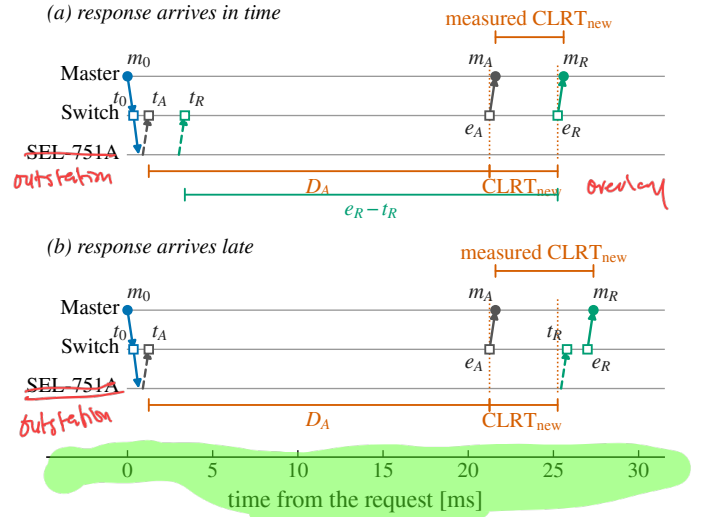


Fig. 4. Release timeline of one READ transaction, with illustrative time coordinates. Both deadlines are computed from the arrival of the acknowledgment t_A . In (a), the response arrives in time and the master observes the configured CLRT_{new} . In (b), it arrives late and leaves when its queue serves it. Filled circles lie on the master-facing link. Open squares are instants inside the switch.

Control Lane. For the OPERATE phase, the framework also holds the command itself on its way to the relay. Because the acknowledgment then no longer marks the start of the work, the framework computes the deadlines when the OPERATE request arrives (Section IV-C).

Both lanes forward the packets they receive at their original size and generate none on either link. As such, the framework changes only when the wire carries the answer of the outstation. *Not clear.*

B. Replacing the CLRT of a Read Transaction

Timing Model. In Figure 4, we show the timeline of one READ transaction. The master sends a request, and the outstation answers it twice: its transport layer first acknowledges the request, and its application later returns the response. We write t_A and t_R for the instants the acknowledgment and the response reach the switch; and e_A and e_R for the instants the switch forwards them. Without the framework, the adversary measures

$$\text{CLRT}_{\text{original}} = t_R - t_A, \quad (1)$$

which is dominated by the execution time of the outstation (Section II). We write

$$D_A = e_A - t_A \quad (2)$$

for the *acknowledgment hold* and refer to $e_R - t_R$ as the *response hold*. The interval the adversary measures then becomes

$$\text{CLRT}_{\text{new}} = e_R - e_A = \text{CLRT}_{\text{original}} + (e_R - t_R) - D_A. \quad (3)$$

Constant Holds. Equation (3) shows why holding both packets for constant times cannot remove the fingerprint. If both holds are constant, the adversary measures the original

Queues and blocker packets not explained. their release and how they don't leave the switch or travel through the wire

interval shifted by their difference. The distribution moves along the time axis and keeps its shape, e.g., a spread of 2.6 ms on our relay, so an adversary who has profiled the device still recognizes it. To remove the fingerprint, the response hold must absorb the variation of $\text{CLRT}_{\text{original}}$.

Dual-Deadline Schedule. To make the response hold absorb that variation, we compute both release deadlines from the same instant t_A . When the acknowledgment arrives, the framework sets the scheduled instants

$$\hat{e}_A = t_A + D_A, \quad \hat{e}_R = t_A + D_A + \text{CLRT}_{\text{new}}, \quad (4)$$

where the configured CLRT_{new} is the interval the operator wants the adversary to measure. Because neither deadline depends on t_R , the term $\text{CLRT}_{\text{original}}$ cancels in (3), and the response hold becomes

$$e_R - t_R = D_A + \text{CLRT}_{\text{new}} - \text{CLRT}_{\text{original}}. \quad (5)$$

The response hold therefore follows the device on every transaction. It is long when the outstation answers quickly and short when the outstation answers slowly.

Availability. The switch can delay a packet but cannot release one it has not received. We call

$$D = D_A + \text{CLRT}_{\text{new}} \quad (6)$$

the *release budget*, and the schedule requires

$$t_R \leq t_A + D. \quad (7)$$

A response that violates (7) arrives after its deadline. The framework marks it late and forwards it as soon as its queue serves it, so the exchange completes with its own timing. The operator chooses D such that late responses are rare (Section IV-D).

Security Argument. Each held packet leaves the switch slightly after its deadline, by a *release tail* ε that the queues of the switch introduce (Section V). Writing $e_A = \hat{e}_A + \varepsilon_A$ and $e_R = \hat{e}_R + \varepsilon_R$, the schedule (4) gives, for every response that satisfies (7),

$$e_R - e_A = \text{CLRT}_{\text{new}} + \varepsilon_R - \varepsilon_A. \quad (8)$$

No term on the right-hand side depends on t_R . Consequently, the execution time of the outstation does not reach the interval the adversary measures. The operator sets the configured value, and the release tails belong to the switch rather than to the device. Nothing in this argument refers to which operation the outstation performed or which device produced the response. A different operation and a different device type are both a different distribution of $\text{CLRT}_{\text{original}}$, and both cancel in the same way. Therefore, a classifier that separates device types or operation types by the CLRT, such as the one of Formby et al. [10], receives the same input distribution for every class, which makes it challenging, if not impossible, to perform better than chance. Pooling does not help the adversary either, since pooling k draws from one distribution yields one distribution for every class, whatever k is. The device can still reach the interval in only two ways, through

release tails that depend on the length of the hold and through responses that violate (7). We evaluate both in Section VI.

Is It Possible to Recover the Execution Time (Adaptive Adversary)? An adaptive adversary knows the framework and the configured CLRT_{new} , but this knowledge reveals only a value the operator chose. The adversary can still measure the intervals the framework does not replace. The request-to-acknowledgment interval carries the acknowledgment latency of the outstation and the hold $D_A + \varepsilon_A$. The acknowledgment latency reflects the transport stack of the outstation rather than its application, e.g., our relay acknowledges READ, SELECT and OPERATE requests within 0.003 ms of one another (Section II). Consequently, any class structure this interval shows under the framework comes from the release logic of the switch, e.g., from the two lanes timing their deadlines from different events, rather than from the execution time of the device. We quantify this residual in Section VI-A2.

C. Replacing the CLRT of a Control Transaction

A select-before-operate control command differs from a poll in one way. The framework also holds the OPERATE request before it reaches the relay, which separates the instant the master issues the command from the instant the relay acts on it. The acknowledgment of the outstation then cannot be the reference instant, because it exists only after the command is released, and a deadline measured from it would carry the command hold into the interval the adversary measures. To remove this dependence, we time the control lane from the request. When the OPERATE arrives at T_0 , the framework releases it toward the relay after a hold J drawn from a configured set, and it schedules the acknowledgment and the response toward the master at

$$\hat{e}_{\text{ack}} = T_0 + D_A, \quad \hat{e}_{\text{or}} = \hat{e}_{\text{ack}} + \text{CLRT}_{\text{new}}. \quad (9)$$

The two offsets are those of the read lane, and one set of deadline registers serves both lanes (Section V).

Security Argument. The adversary measures $O = e_{\text{or}} - e_{\text{ack}} = \text{CLRT}_{\text{new}} + \varepsilon_{\text{or}} - \varepsilon_{\text{ack}}$, in which neither the execution time of the relay nor the command hold J appears. As such, the framework can vary the instant the relay acts on a command while the interval the adversary measures stays fixed, and the adversary cannot subtract a known offset to recover either quantity. As in the read lane, the schedule requires the response of the relay to reach the switch before $\hat{e}_{\text{or}}$, i.e., within D of the request, with the command hold J included.

D. Overhead Argument

Obfuscation is only useful if an operator can afford it. We now argue why the framework introduces negligible overhead (RO3) in any deployment.

Bandwidth Overhead. As shown in Figure 3, the framework forwards the packets it receives at their original size. The blocker packets that sustain a hold recirculate on internal loopback ports and never reach a cable (Section V). Consequently, the framework adds no frame and no byte to either

link. Obfuscation that pads packets or injects dummy ones pays a bandwidth cost that this design does not have [4], [7].

Latency Bound. We delay two packets per transaction. The acknowledgment leaves $D_A + \varepsilon_A$ after it arrives. A response that satisfies (7) leaves at $t_A + D + \varepsilon_R$, so by (5) its added delay, $D - \text{CLRT}_{\text{original}} + \varepsilon_R$, never exceeds $D + \varepsilon_R$ and shrinks as the outstation answers more slowly. A response that violates (7) leaves as soon as its queue serves it. The operator therefore bounds every added delay by choosing the release budget D , and the release tail is the only term the operator does not set.

Protocol Timers. Three timers of the two endpoints can observe these delays. The TCP stack of the master retransmits a request whose acknowledgment does not arrive within its retransmission timeout τ_m . The TCP stack of the outstation retransmits a response the master does not acknowledge within its own timeout τ_o . The relay cancels a selected control point whose OPERATE does not follow within the select-to-operate timeout τ_s , 1.0 s by default on the SEL-751A [19]. The acknowledgment hold delays the first, the response hold delays the second, and the hold on the SELECT response delays the third. Consequently, the framework triggers none of them as long as

$$D + \varepsilon < \min(\tau_m, \tau_o, \tau_s). \quad (10)$$

Condition (10) holds for any deployment whose outstation answers well within its timers, because the budget only needs to cover the response time of the outstation. RFC 6298 recommends a lower bound of one second on the retransmission timeout [20]. On our testbed, the master repeats an unacknowledged request after 201 ms and the relay repeats an unacknowledged response after about 3 s, while a 24 ms budget covers 99.9% of the responses of the relay (Section VI-B).

Security-Performance Trade-off Argument. The release budget D sets both sides of a trade-off. A larger budget places more of the responses of the outstation on the schedule, so fewer late responses carry their own timing, and it adds more delay to every transaction. Within a fixed budget, the split between D_A and the configured CLRT_{new} costs nothing, and the operator can place the released interval anywhere below D without changing the delay. We set CLRT_{new} to a constant, because a constant removes the variation of the device entirely and an operator can state and check it. The schedule can carry any other value, e.g., one drawn from a distribution, without change.

V. IMPLEMENTATION

To evaluate the framework, we implemented it on an Intel Tofino-1 switch and placed the switch between a DNP3 master and a physical protective relay. In this section, we first describe how the switch holds a packet until its deadline. Then we describe how it processes a transaction and the testbed we use.

A. Holding Packets with Queues

A P4 program runs to completion on each packet, and it can read a clock. However, it can neither sleep nor schedule

a callback, so it cannot wait for a deadline by itself. The switch does offer several queues per port and a strict-priority scheduler [15]. Because a scheduler decides which packets do not leave, we hold a packet with the scheduler rather than with a timer.

Blocker Queues. As shown in Figure 3, the read lane uses four queues on one internal port. Two queues carry the packets we delay, the acknowledgment and the response. Above each of them sits a *blocker queue* of higher priority, which carries small packets the switch generates itself. While blocker packets keep arriving, the packet beneath them is never served, and the hold ends when its blocker queue runs dry. The response has its own held queue and its own blocker queue, so the switch can release the acknowledgment and the response independently. Blocker packets recirculate on an internal loopback port and never reach a cable, so neither endpoint can observe them. Each blocker packet carries a tag naming its transaction and lane, so a leftover blocker packet cannot delay a later transaction.

Deadline Check. No counter drives the hold. When the acknowledgment arrives, the program stores an absolute release instant for each packet, quantized to a 256 ns grid. Each of the 64 recirculating blocker packets compares the current timestamp against that instant on every pass. The program regenerates the blocker packet while the deadline lies in the future and drops it once the deadline passes. Consequently, the deadline decides when the held packet leaves, and the blocker packets only keep the queue non-empty until then.

Release Tail. The held packet does not leave at the deadline itself. Blocker packets already queued when the deadline passes are served once more before the queue runs dry, and only then does the held packet leave. This drain time is the release tail ε of Section IV-B, and we measure it in Section VI-B2.

B. Processing a Transaction

Read Transaction. When a request arrives, a table classifies it by its function code, and the program forwards it at once and records the transaction in flight. When the acknowledgment arrives at t_A , a median of 0.56 ms later on our relay, the program writes both release instants from that one timestamp, as (4) requires. It writes them only from an unarmed state, so a duplicate acknowledgment cannot arm a second deadline. A response that arrives before its deadline waits in its held queue, and the two packets leave at their scheduled instants. A response that arrives after its deadline is marked late and enters the same held queue, from which it leaves when the queue next serves it. Releasing the response clears the transaction.

Control Transaction. The control lane uses a second loopback port with one blocker queue above the held OPERATE request, on its way to the relay. The acknowledgment and the response of the OPERATE then pass through the read-lane queues, which release them at the instants of (9). Placing the OPERATE on its own loopback port gives it a scheduling domain of its own, so the higher-priority read-lane queues cannot starve it. Both lanes share one set of deadline registers,

so the program protects one transaction at a time. A second transaction that would arm while one is in flight is forwarded unprotected rather than disturbing the deadline in progress.

Failing Open. Every path through the program ends in a forwarding decision. A packet outside the protected session, an unrecognized request, an acknowledgment without an armed transaction and a response whose transaction has already been released all leave unchanged. A budget on blocker passes bounds every hold, so once a blocker queue spends its budget the held packet leaves and its state clears. From this budget, the control plane derives the longest hold the switch can sustain, a horizon H of 30.8 ms. With the framework disabled, the same program forwards every packet at once, which gives the Timing OFF case.

C. Testbed

Switch Program. The program has about 1,560 lines of P4₁₆ [14] for the Tofino Native Architecture, compiled with the Barefoot SDE 9.13. It occupies all twelve ingress stages and six egress stages of one pipeline, with 112 tables. Its state lives in one-entry registers, each accessed once per packet. One 25 Gb/s port connects the master and one 1 Gb/s port connects the relay. Two further ports are loopbacks without cables, one for the read lane and one for the control lane.

Control Plane. A control plane of about 890 lines of Python brings up the ports and the queues, configures the packet generator, and writes the release policy into runtime tables and reads it back. An operator therefore changes the policy by writing table entries rather than loading a new program. The control plane refuses any acknowledgment hold above 40 ms.

Physical Device. The outstation is an SEL-751A feeder protection relay, one of the DNP3 device classes that Formby et al. fingerprint [10].

DNP3 Master. The master issues READ polls and select-before-operate controls through a guarded driver. Every control operation addresses an isolated control point, so no operation in the evaluation moves a breaker.

VI. EVALUATION

In this section, we perform security and performance evaluation of the framework. The security evaluation covers RO1 and RO2 (Section VI-A), and the performance evaluation covers RO3 (Section VI-B).

Evaluation Cases. We compare two cases on the testbed of Section V. In *Timing OFF*, the switch forwards the acknowledgment and the response of the outstation as they arrive. In *Obfuscated*, the framework holds and releases them on the schedule of Section IV. Both cases run the same switch program and differ only in the release schedule. We timestamp every packet on the master-facing link, where the adversary sits, so every interval we report is one the adversary can measure.

Datasets. We use three datasets. In the main campaign, we collected 63,360 DNP3 exchanges in 22 grouped runs, half in each case, interleaved over five hours so that drift

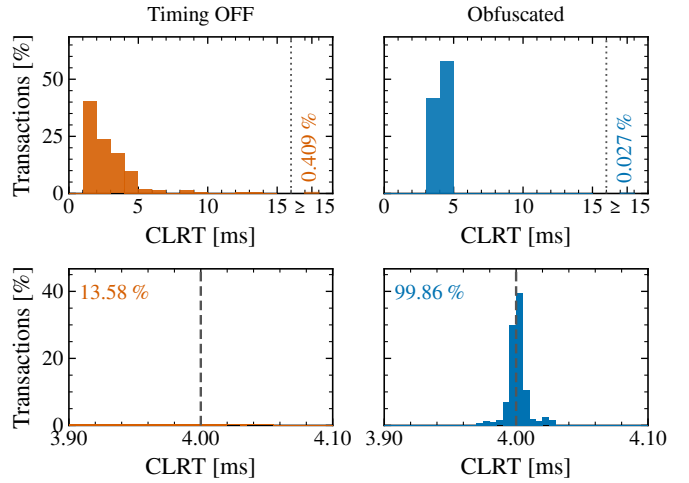


Fig. 5. READ CLRT measured on the master-facing link, in both cases (columns) and at two scales (rows). The upper row uses 1 ms bins, and its last bin holds every interval of 15 ms or more. The lower row uses 0.005 ms bins within 0.10 ms of the configured $CLRT_{new}$ (dashed). Both rows give the percentage of all 26,400 exchanges of each case.

cannot separate the two cases. We set D_A to 20 ms and the configured $CLRT_{new}$ to 4 ms, and the control lane draws J from $\{2, 6, 12\}$ ms. A policy sweep installs 16 release policies, and a budget sweep installs 18 release budgets three times each, 27,111 exchanges in total. Every request in the main campaign was answered, and no exchange was excluded.

Adversary Models. Following Formby et al. [10], the adversary classifies each exchange into one of three operations, READ, SELECT and OPERATE, from its CLRT and its request-to-acknowledgment interval, with a random forest. We report balanced accuracy, for which chance is 0.333, and hold out one grouped run at a time. We also estimate the mutual information between the CLRT and the operation against a null built by shuffling the labels within each run 1,000 times, which measures the information available to any classifier.

A. Security Evaluation

Security evaluation focuses on the effectiveness of (i) the timing replacement (RO1) and (ii) the resulting resistance to fixed and adaptive fingerprinting (RO2).

1) *Effectiveness of Timing Replacement: Effectiveness in RO1.* In Figure 5, we show the READ CLRT the adversary measures in both cases. Under Timing OFF, the interval spans 1.01 to 83.46 ms, with peaks near 1.2, 2.1 and 4.0 ms that reflect how long the relay takes to answer. Under the framework, 99.9% of READ exchanges fall within 0.10 ms of the configured 4 ms. From the figure, we can see that the medians move from 2.116 ms for READ and 2.050 ms for SELECT to 4.000 ms, while the interquartile ranges fall from 2.778 and 2.697 ms to 0.006 ms. The standard deviation falls from 2.609 to 0.628 ms for READ and from 2.267 to 0.024 ms for SELECT. A defense that only shifted the interval would keep the Timing OFF spread, since shifting a distribution leaves its variance unchanged. The framework

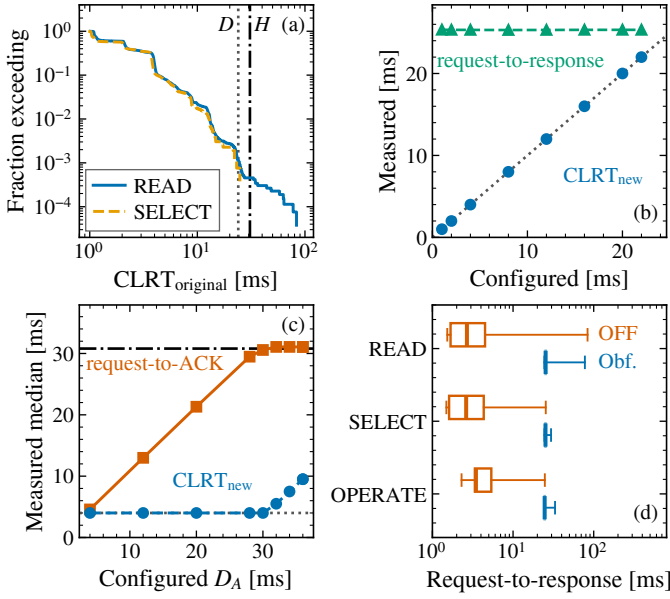


Fig. 6. Release policy and cost. (a) Timing OFF response time of the read lane against the budget D and the horizon H . (b) At $D = 24$ ms, the measured CLRT_{new} against the configured one, with the response time of the exchange. (c) The measured request-to-acknowledgment interval against the configured D_A . (d) End-to-end response time per class in both cases.

therefore replaces the interval instead of moving it, and it does so in each of the 22 runs.

Control Lane. The control lane achieves the same replacement for OPERATE, although the framework holds the command itself for a varying J before it reaches the relay. The OPERATE median moves from 2.937 ms under Timing OFF to 4.000 ms, and its interquartile range falls from 2.827 to 0.006 ms. As (9) predicts, J does not appear in the interval. In the lower-right panel of Figure 8, we show that the OPERATE interval follows the configured CLRT_{new} and ignores every other configured offset.

Programmable Policy. Based on one budget, an operator can place the released interval anywhere within it. In Figure 6(b), we show that at a fixed budget of 24 ms, configured values from 1 to 22 ms produce measured medians within $2 \mu\text{s}$ of the setting, while the end-to-end response time stays between 25.30 and 25.34 ms. Consequently, the operator sets the interval the adversary measures and the delay of the exchange independently. In Figure 6(c), we show the upper end of the range. The acknowledgment hold follows its setting up to 30 ms, where the measured median is 30.571 ms and the released interval still sits at 4.001 ms. Beyond that setting the switch can no longer sustain the hold, and the released interval rises to 9.507 ms at a 36 ms hold.

2) *Effectiveness against Fingerprinting: Effectiveness in RO2.* Under Timing OFF, the CLRT separates the three operations, with medians of 2.116, 2.050 and 2.937 ms, while their request-to-acknowledgment medians of 0.555, 0.555 and 0.558 ms barely differ. In Figure 7(a), we show that a fixed adversary trained on Timing OFF traffic reaches 0.745 balanced

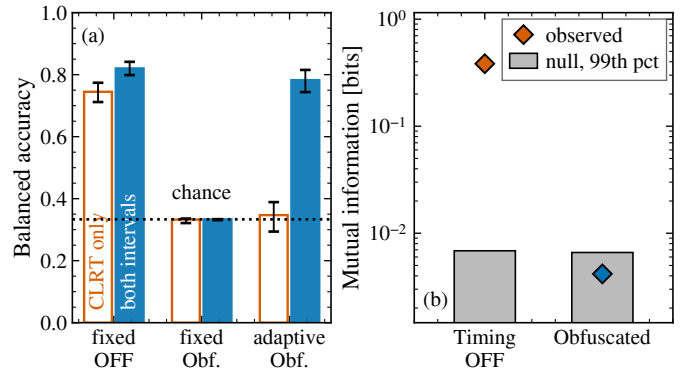


Fig. 7. Fingerprinting the three operations, with one grouped run held out at a time. (a) Balanced accuracy of a fixed adversary tested on Timing OFF and on Obfuscated traffic, and of an adaptive adversary retrained on Obfuscated traffic. Whiskers span the 22 held-out runs. (b) Mutual information between the CLRT and the operation, against a within-run permutation null.

accuracy from the CLRT alone and 0.820 from both intervals. Applied to obfuscated traffic, the same adversary falls to 0.333 in both cases, i.e., chance. In Figure 7(b), we show the reason. The mutual information between the CLRT and the operation falls from 0.383 bits, far above its permutation null ($p = 0.001$), to 0.004 bits, inside it ($p = 0.084$). This is because the framework removes the execution time from the interval (Section IV-B), so the CLRT carries no class information for any classifier to find.

Pooling Adversary. Formby et al. build a signature from many observations rather than one [10]. We therefore give the adaptive adversary that advantage. The adversary pools k exchanges of the same operation, summarizes each interval by its mean, spread and extremes, and retrained on obfuscated traffic. Under Timing OFF, pooling makes the CLRT highly identifying, and balanced accuracy rises from 0.747 for one exchange to 0.988 for twenty. Under the framework, the same adversary stays at chance, with 0.347 for one exchange and 0.341 for twenty. This is because pooling draws from one distribution yields one distribution for every operation, as argued in Section IV-B.

Adaptive Adversary. The adaptive adversary can still read the request-to-acknowledgment interval, which the framework does not replace. Retrained on both intervals, the adversary reaches 0.782 balanced accuracy from one exchange and 0.942 from five. The source of this accuracy is the release logic of the switch rather than the execution time of the relay. Because the read lane times its deadlines from the acknowledgment and the control lane from the request, the adversary identifies OPERATE in 0.940 of cases. READ and SELECT separate through request-to-acknowledgment medians of 21.336 and 21.239 ms, a gap that appears only under the framework, since the relay acknowledges both at a median of 0.555 ms under Timing OFF.

Discussion. Operations of one device differ in execution time, e.g., by 0.8 ms between a poll and a control command on our relay, and device types differ in the same way [10], [13].

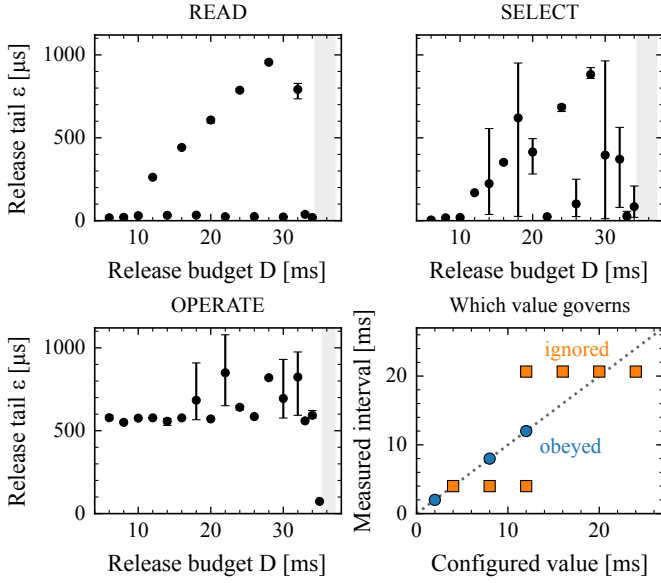


Fig. 8. Release tail ϵ against the budget D for each class, over three installs of each setting. Whiskers span the installs. In the shaded region, the switch cannot sustain the hold. OPERATE is timed from the request, so its ϵ is the arrival of the acknowledgment beyond $T_0 + D_A$. Lower right, the master-visible OPERATE interval against each configured offset that could set it. Only the configured CLRT_{new} does.

Because the framework removes the execution time whatever its source (Section IV-B), the same argument applies to device types. Behind the switch, every outstation whose responses arrive within the budget presents the same released interval. The residual that an adaptive adversary exploits lies in the release logic of the switch, which the operator configures.

B. Performance Evaluation

The framework introduces overhead in two forms, bandwidth and latency, and the latency can interact with the timers of TCP and DNP3. We evaluate each against RO3.

1) *Bandwidth and Latency Overhead: Effectiveness in RO3.* We observe no bandwidth overhead. Each exchange carries 3.017 frames and 272.308 bytes on the master-facing link in both cases, so the framework moves packets in time without adding a frame or a byte. In Figure 6(d), we show the end-to-end response time, which the framework raises by 22.7 ms for READ, 22.6 ms for SELECT and 21.2 ms for OPERATE. Each increase stays below the 24 ms budget, as the latency bound of Section IV-D predicts, because the hold absorbs the response time of the outstation.

2) *Release Accuracy: Release Tail.* Each held packet leaves after its deadline by the release tail ϵ , which adds to the configured hold. Because the switch timestamps no departure, we obtain ϵ from the two cases, as the obfuscated request-to-acknowledgment median minus the Timing OFF median minus the configured hold. In the main campaign, ϵ is 0.78 ms at the 24 ms budget, already included in the latency above. In Figure 8, we show ϵ across the budget sweep. For READ, it stays between 18 and 38 μ s at ten budgets and reaches 0.96 ms

at the other six, and for OPERATE it stays between 0.55 and 0.85 ms. For every class and every budget the switch can sustain, the median release tail stays below 1 ms, two orders of magnitude below the timers of Section IV-D.

3) *Impact on Protocol Timers: Retransmission.* On the master-facing captures, neither case carried a TCP retransmission, a reset or a duplicate acknowledgment. The longest the master waited under the framework was 29.2 ms for an acknowledgment and 77.7 ms for a response. In a separate test that withheld every acknowledgment of one request, the master repeated the request after 201 ms, about seven times the longest wait.

Select Validity. The master issues its OPERATE 0.19 ms after the SELECT response reaches it, so the hold on that response consumes part of the select-to-operate timeout. No obfuscated OPERATE was refused, and all 5,280 SELECT and OPERATE exchanges returned a success status.

Coverage. In Figure 6(a), we show the Timing OFF response time of the read lane against the budget. At 24 ms, the budget covers 99.900% of exchanges. The remaining one in a thousand arrives too late to be held and leaves with its own timing, which gives the largest released READ interval of 57.182 ms. A larger budget covers more of these exchanges at the cost of more delay, as argued in Section IV-D.

C. Limitations

Three limitations bound these results.

Single Device. We evaluate one SEL-751A behind one Tofino-1 switch. Our security argument in Section IV-B does not depend on the device, and the three operations of the relay, whose execution times differ, exercise it. Measuring the released interval across outstations from several vendors, as Formby et al. and Lin et al. did for the original interval [10], [13], would test the argument for device types directly, and we leave it to future work.

Unobserved Relay-Facing Link. We capture the master-facing link, where the adversary sits. As such, the command hold J and the release toward the relay are configured values rather than measured ones.

Retransmitted Commands. After releasing an OPERATE, the control lane keeps its generation until the next SELECT and drops a retransmission that carries it. A command lost on the relay-facing link is therefore not repaired by its own retransmission, and extending the control lane to forward such a retransmission is future work.

VII. RELATED WORK

Device Fingerprinting. Devices can be identified from what they emit, without access to the payload. Kohno et al. fingerprint a remote host from the skew of its clock in TCP timestamps [21], and GTID identifies a device and its type from the timing signature of its packets [22]. These works define the threat we address, and each of them reads a different observable. Compared to them, we target the interval that the cross-layer response-time method of Formby et al. classifies by, and we replace that interval on the master-facing link.

ICS and Power-System Fingerprinting. Formby et al. brought fingerprinting to control systems with the cross-layer response time and the physical operation time of DNP3 outstations [10], and Gu et al. added device physics as a feature [23]. Pospisil et al. identify PLC models passively from the time delay in their dataflow [24], which shows that execution time still identifies industrial devices under other protocols. The reconnaissance these techniques serve enables false data injection and process-control attacks [17], [25]. Consequently, the interval our framework replaces is the observable this line of work relies on.

Anti-Reconnaissance in Power Grids. The defenses closest to ours change what the traffic says and which nodes it reaches. DefRec virtualizes physical functions and crafts decoy data for virtual nodes [13], RAINCOAT randomizes data acquisition and injects decoy measurements [26], and a companion testbed evaluates such approaches on grid infrastructure [27]. These approaches are effective against reconnaissance of network and physical configurations. However, they build their protection above the packets an outstation sends, so the execution time of a real device still travels with its answer. Our framework, on the other hand, works below the semantics they reshape and changes when the wire carries that answer, which makes the two complementary.

General Traffic Obfuscation. Traffic morphing transforms the packet-size distribution of one class into another [3], while Dyer et al. show that per-packet padding and fragmentation still leave exploitable features [4]. Tamaraw fixes packet size and rate at a bandwidth cost [6], and WTF-PAD pads inter-packet gaps adaptively [28]. These defenses are effective against the flow-level features they target, and the attacks that follow them have become correspondingly strong [5], [29]. The same size-and-timing channel identifies smart-home devices under encryption, where shaping is the countermeasure [30], and Pacer and NetShaper shape the traffic of a tenant to a schedule that bounds side-channel leakage in the cloud [31], [32]. Pacer and NetShaper each shape from a host or a hypervisor that can add cover traffic, and the smart-home setting assumes an encrypted payload. Compared to these settings, ours carries a plaintext protocol whose leaking feature is one interval inside a single request-response exchange, and no endpoint can be modified to help. Consequently, our framework adds no byte and runs entirely in the network.

Programmable Data-Plane Traffic Transformation. P4 defines the programmable parser and match-action model we use [14]. PIFO defines programmable scheduling at line rate [33], and SP-PIFO approximates it with the strict-priority queues of today’s switches [15], which is the primitive our blocker queues use. Ditto pads packets and injects chaff to a fixed pattern at line rate, and it carries up to 70 Gbps of production traffic on a 100 Gbps link [7]. Minos morphs and schedules traffic on a programmable switch at low cost [8], and Securitas combines fragmentation and insertion across several data planes [9]. NetHide obfuscates topology in the data plane against link-flooding reconnaissance [34]. Closest

to our setting, Hu et al. use programmable switches to enhance the security of industrial protocols [35]. These systems reshape which packets a link carries and how large they are. Our framework, on the other hand, addresses one interval inside a transaction, and it holds packets rather than adding or reshaping them.

DNP3 Timing and Security. East et al. catalogue attacks on DNP3 and the absence of encryption that makes its semantics visible [16], and specification-based and state-based intrusion detection has been adapted to DNP3 [18], [36]. These defenses inspect what a packet says, and none of them changes the timing an adversary sees. On the control path, Formby et al. read the physical operation time from an event timestamp inside the application payload [10], which is a different channel from the packet timing our framework replaces.

VIII. CONCLUSION

Instead of padding packets or adding dummy ones, we propose an in-network framework that holds the packets carrying the timing of an ICS device and releases them on a schedule the operator sets. Because both release deadlines are computed from one instant, the execution time of the device cancels from the interval an adversary measures, which is the interval that cross-layer response-time fingerprinting relies on. The framework runs without encryption and leaves the master, the outstation and DNP3 unchanged.

On a physical SEL-751A relay, the released interval settles on the configured value, its interquartile range falls from 2.778 to 0.006 ms, and the operator can place it anywhere from 1 to 22 ms. A classifier trained on Timing OFF traffic falls from 0.745 balanced accuracy to chance, and an adaptive adversary that pools twenty exchanges, as fingerprinting methods do, stays at chance, where Timing OFF traffic lets it reach 0.988. The framework adds no frame and no byte, and it raises the median response time by 22.7 ms, well within every timer of TCP and DNP3. In future work, we will evaluate the framework on outstations from several vendors and on intervals beyond the two we replace.

OPEN SCIENCE

We reproduce every number and every campaign figure in this paper from the raw captures with one script, in a pinned environment archived with the extraction and analysis code. A hash manifest covers each published artefact, and a publication gate compares a fresh rebuild against what the paper reports, so a discrepancy fails rather than passing silently. We also publish the switch program that ran, with its compilation record. The firmware of the relay and its vendor documentation are not ours to distribute. On acceptance, we release the captures, the analysis pipeline and the switch program under an open license.

ETHICS CONSIDERATIONS

This work required no human subjects and no personal data. We ran every experiment on a laboratory testbed we

own, namely one SEL-751A protective relay, one Intel Tofino-1 switch and one master host, on an isolated network with no connection to any operational system. We contacted no production grid, no utility network and no third-party device.

The relay is physical equipment whose outputs can operate a breaker. Every select-before-operate exchange in the evaluation therefore addresses isolated control points through a guarded driver. No control operation in this paper moved a breaker or energized a circuit, and each OPERATE is a protocol exchange the relay accepted and acknowledged.

The framework is defensive. It removes a timing feature that an adversary uses for reconnaissance, and it neither creates a new attack nor discloses a vulnerability in the relay, the switch or the DNP3 protocol.

REFERENCES

- [1] R. M. Lee, M. J. Assante, and T. Conway, "Analysis of the Cyber Attack on the Ukrainian Power Grid," Electricity Information Sharing and Analysis Center (E-ISAC) and SANS Institute, Defense Use Case, Mar. 2016.
- [2] R. Langner, "Stuxnet: Dissecting a Cyberwarfare Weapon," *IEEE Security & Privacy*, vol. 9, no. 3, pp. 49–51, 2011.
- [3] C. V. Wright, S. E. Coull, and F. Monrose, "Traffic Morphing: An Efficient Defense Against Statistical Traffic Analysis," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2009.
- [4] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Peek-a-Boo, I Still See You: Why Efficient Traffic Analysis Countermeasures Fail," in *2012 IEEE Symposium on Security and Privacy (S&P)*, 2012.
- [5] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018.
- [6] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, "A Systematic Approach to Developing and Evaluating Website Fingerprinting Defenses," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2014, pp. 227–238.
- [7] R. Meier, V. Lenders, and L. Vanbever, "Ditto: WAN Traffic Obfuscation at Line Rate," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2022.
- [8] Z. Wang, Q. Li, G. Xie, D. Zhao, K. Li, Z. Fan, L. Ma, and Y. Jiang, "Minos: A Lightweight and Dynamic Defense against Traffic Analysis in Programmable Data Planes," in *Proceedings of the 2025 USENIX Annual Technical Conference (ATC)*, 2025.
- [9] G. Xie, Q. Li, Z. Shi, G. Antichi, Y. Zhu, K. Li, C. Weng, S. Miano, Y. Jiang, and M. Xu, "Defending against Traffic Analysis Attacks with Flexible In-Network Obfuscation," in *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2026, pp. 2043–2063.
- [10] D. Formby, P. Srinivasan, A. Leonard, J. Rogers, and R. Beyah, "Who's in Control of Your Control System? Device Fingerprinting for Cyber-Physical Systems," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2016.
- [11] IEEE, "IEEE Standard for Electric Power Systems Communications—Distributed Network Protocol (DNP3)," 2012.
- [12] W. Eddy, "Transmission Control Protocol (TCP)," 2022.
- [13] H. Lin, J. Zhuang, Y.-C. Hu, and H. Zhou, "DefRec: Establishing Physical Function Virtualization to Disrupt Reconnaissance of Power Grids' Cyber-Physical Infrastructures," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2020.
- [14] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: Programming Protocol-Independent Packet Processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.
- [15] A. G. Alcoz, A. Dietmüller, and L. Vanbever, "SP-PIFO: Approximating Push-In First-Out Behaviors using Strict-Priority Queues," in *Proc. 17th USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2020, pp. 59–76.
- [16] S. East, J. Butts, M. Papa, and S. Sheno, "A Taxonomy of Attacks on the DNP3 Protocol," in *Critical Infrastructure Protection III (IFIP AICT, Vol. 311)*. Springer, 2009, pp. 67–81.
- [17] Y. Liu, P. Ning, and M. K. Reiter, "False Data Injection Attacks against State Estimation in Electric Power Grids," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2009, pp. 21–32.
- [18] H. Lin, A. Slagell, C. D. Martino, Z. Kalbarczyk, and R. K. Iyer, "Adapting Bro into SCADA: Building a Specification-Based Intrusion Detection System for the DNP3 Protocol," in *Proceedings of the Eighth Annual Cyber Security and Information Intelligence Research Workshop (CSIIRW)*, 2013.
- [19] Schweitzer Engineering Laboratories, *SEL-751A Feeder Protection Relay Instruction Manual*, 2013, pM751A-01-NB, 20130329 printing; settings table, STIME01 select/operate time-out.
- [20] V. Paxson, M. Allman, J. Chu, and M. Sargent, "Computing TCP's Retransmission Timer," 2011.
- [21] T. Kohno, A. Broido, and K. C. Claffy, "Remote Physical Device Fingerprinting," *IEEE Transactions on Dependable and Secure Computing*, vol. 2, no. 2, pp. 93–108, 2005.
- [22] S. V. Radhakrishnan, A. S. Uluagac, and R. Beyah, "GTID: A Technique for Physical Device and Device Type Fingerprinting," *IEEE Transactions on Dependable and Secure Computing*, vol. 12, no. 5, pp. 519–532, 2015.
- [23] Q. Gu, D. Formby, S. Ji, H. Cam, and R. Beyah, "Fingerprinting for Cyber-Physical System Security: Device Physics Matters Too," *IEEE Security & Privacy*, vol. 16, no. 5, pp. 49–59, Sep. 2018.
- [24] O. Pospisil and R. Fudjak, "Identifying Industry Devices via Time Delay in Dataflow," in *Proceedings of the 2023 13th International Conference on Communication and Network Security (ICCNS)*. ACM, 2023, pp. 173–177.
- [25] A. A. Cárdenas, S. Amin, Z.-S. Lin, Y.-L. Huang, C.-Y. Huang, and S. Sastry, "Attacks Against Process Control Systems: Risk Assessment, Detection, and Response," in *Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security (ASIACCS)*, 2011, pp. 355–366.
- [26] H. Lin, Z. T. Kalbarczyk, and R. K. Iyer, "RAINCOAT: Randomization of Network Communication in Power Grid Cyber Infrastructure to Mislead Attackers," *IEEE Transactions on Smart Grid*, vol. 10, no. 5, pp. 4893–4906, 2019.
- [27] H. Lin, B. Shrestha, and Y.-C. Hu, "Cyber-Physical Testbed: Case Study to Evaluate Anti-Reconnaissance Approaches on Power Grids' Cyber-Physical Infrastructures," in *Proc. Workshop on Learning from Authoritative Security Experiment Results (LASER)*, 2020.
- [28] M. Juarez, M. Imani, M. Perry, C. Diaz, and M. Wright, "Toward an Efficient Website Fingerprinting Defense," in *Computer Security – ESORICS 2016*, 2016.
- [29] A. Panchenko, F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, and K. Wehrle, "Website Fingerprinting at Internet Scale," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2016.
- [30] N. Apthorpe, D. Y. Huang, D. Reisman, A. Narayanan, and N. Feamster, "Keeping the Smart Home Private with Smart(er) IoT Traffic Shaping," *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2019.
- [31] A. Mehta, M. Alzayat, R. D. Viti, B. B. Brandenburg, P. Druschel, and D. Garg, "Pacer: Comprehensive Network Side-Channel Mitigation in the Cloud," in *Proceedings of the 31st USENIX Security Symposium*, 2022.
- [32] A. Sabzi, R. Vora, S. Goswami, M. Seltzer, M. Lécuyer, and A. Mehta, "NetShaper: A Differentially Private Network Side-Channel Mitigation System," in *Proceedings of the 33rd USENIX Security Symposium*, 2024.
- [33] A. Sivaraman, S. Subramanian, M. Alizadeh, S. Chole, S.-T. Chuang, Agrawal, Anurag, H. Balakrishnan, T. Edsall, S. Katti, and N. McKeown, "Programmable Packet Scheduling at Line Rate," in *Proc. ACM SIGCOMM Conference*, 2016, pp. 44–57.
- [34] R. Meier, P. Tsankov, V. Lenders, L. Vanbever, and M. Vechev, "NetHide: Secure and Practical Network Topology Obfuscation," in *Proc. 27th USENIX Security Symposium*, 2018, pp. 693–709.
- [35] Z. Hu, H. Lin, L. Waind, Y. Qu, G. Chen, and D. Jin, "Industrial Network Protocol Security Enhancement Using Programmable Switches," in *Proceedings of the IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm)*, 2023.
- [36] I. N. Fovino, A. Carcano, T. D. L. Murel, A. Trombetta, and M. Masera, "Modbus/DNP3 State-Based Intrusion Detection System," in *2010 24th IEEE International Conference on Advanced Information Networking and Applications (AINA)*, 2010, pp. 729–736.